Glaukopis: Cyber Threat Intelligence Language Model Training: Optimizing Domain Specific LLM Performance

Peter J. Worth Jr., Dr. Ionut Cardei & Md Tanvirul Alam
Department of Electrical Engineering and Computer Science
Florida Atlantic University, Boca Raton, FL, USA

Abstract

Large language models (LLMs) are increasingly applied to cybersecurity tasks ranging from vulnerability analysis to cyber threat intelligence (CTI) reporting. However, generic foundation models often lack the domain knowledge, calibration, and reasoning needed for high-stakes CTI workflows, while frontier models raise cost, latency, and privacy concerns. Recent work from IBM on CyberPal.AI and its follow-on line "Toward Cybersecurity-Expert Small Language Models" demonstrates that expert-curated instruction fine-tuning (IFT) can transform small language models (SLMs) into capable cybersecurity assistants when paired with high-quality, domain-specific instruction datasets such as SecKnowledge and SecKnowledge 2.0.[42, 43]

In parallel, the CTI community has developed rich knowledge bases (MITRE ATT&CK, CVE, CAPEC, CWE, ENGAGE) and domain-specific benchmarks such as CTIBench and, most recently, AthenaBench—a dynamic, multi-task CTI benchmark that integrates live data sources and emphasizes applied security reasoning.[70, 2, 3, 1, 6, 5]

This paper surveys the research foundations underlying an ongoing effort at Florida Atlantic University and Athena Labs (Athena Security Group) to build instruction-tuned cybersecurity SLMs using a template-based IFT data generation system grounded in a CTI knowledge graph. We (1) review instruction fine-tuning and SLMs with emphasis on CyberPal and SecKnowledge; (2) describe the role of CTI knowledge bases and knowledge graphs in structuring cyber reasoning tasks; (3) situate Athena's template-based IFT generator within this landscape; and (4) analyze the state of cybersecurity LLM benchmarking, with particular focus on AthenaBench as an emerging standard for CTI evaluation. We conclude by outlining research directions for aligning template-generated IFT corpora, cybersecurity SLMs, and dynamic CTI benchmarks.

1 Introduction

LLMs have rapidly become integral to many security workflows: summarizing threat reports, mapping indicators to MITRE ATT&CK, triaging vulnerabilities, and assisting with detection engineering. Surveys of LLMs for security show use cases spanning vulnerability detection, malware analysis, CTI, phishing detection, and SOC automation, but also highlight persistent issues: hallucinations, shallow pattern matching rather than causal reasoning, and limited ability to operate on up-to-date threat information.[42, 46]

Two trends have emerged in response:

- **Domain-specific models and datasets.** Models such as SecureBERT and CySecBERT pretrain or adapt transformers on cybersecurity corpora, improving representation quality for CTI and related tasks. Instruction-tuned cybersecurity assistants such as Lily-Cybersecurity-7B and IBM's CyberPal models further specialize LLMs for interactive security workflows using cyber-focused instruction–response pairs.[42]
- **Domain-specific benchmarks.** Benchmarks such as CyberMetric, CyberBench, SecEval, CTIBench, and SEvenLLM-Bench evaluate security knowledge, reasoning, and incident analysis in LLMs.[6, 34, 46]

However, many existing benchmarks are static and quickly fall out of sync with the evolving threat landscape. AthenaBench addresses this by introducing a *dynamic CTI benchmark* that continuously draws from live sources (MITRE ATT&CK, NVD, current APT reports) to construct tasks targeting CTI knowledge, vulnerability severity, root cause mapping, threat actor attribution, mitigation strategies, and attack technique extraction.[5]

In parallel, the IFT design document by Cardei describes an *IFT Dataset Generation System* that compiles high-level templates into (instruction, question, answer) triples grounded in an ASG Neo4j CTI graph built from MITRE and other

CTI sources.[13] This system is intended to produce large, structured, and incrementally refreshable cybersecurity IFT corpora for training SLMs specialized to CTI workflows.

This paper positions that work in its research context:

- Section 3 reviews instruction fine-tuning and small language models, emphasizing IBM's CyberPal line.
- Section 7 discusses CTI, CTI knowledge bases, and the role of knowledge graphs and templates in generating IFT data.
- Section 8 surveys cybersecurity LLM benchmarks, highlighting AthenaBench and its relationship to CTIBench and other efforts.
- Section 9 connects these strands to the Athena IFT generator design and outlines research opportunities.

2 Instruction Fine-Tuning: History and Context

Instruction fine-tuning (IFT) has emerged as a foundational paradigm for aligning large language models (LLMs) with human intent and task-oriented usage. Rather than treating language modeling as an end in itself, IFT reframes downstream interaction as the problem of mapping natural-language instructions to correct, task-specific outputs. Although early work on task fine-tuning existed prior to large-scale transformers, the modern formulation of instruction fine-tuning coalesced through three seminal research efforts between 2021 and 2022. Together, these works established the empirical phenomenon, the alignment pipeline, and the scalability properties that underpin nearly all contemporary instruction-tuned models, including domain-specialized systems such as CyberPal.AI and the Athena CTI instruction framework.

2.1 FLAN: Instruction Tuning as a Generalization Paradigm

The first widely recognized formulation of instruction fine-tuning was introduced by Wei et al. in *Finetuned Language Models Are Zero-Shot Learners* (FLAN) [84]. This work demonstrated that large pretrained language models, when fine-tuned on a sufficiently diverse collection of tasks expressed uniformly as natural-language instructions, exhibit strong zero-shot generalization to unseen tasks.

FLAN unified dozens of existing NLP datasets by converting each task into multiple instruction templates (e.g., question answering, sentiment analysis, textual entailment, translation). The authors showed that instruction formatting itself—explicitly describing the task in natural language—was a critical factor in enabling generalization, often rivaling or surpassing few-shot prompting on much larger models.

The central insight of FLAN is that *instruction-following is a learnable capability* rather than an emergent property of scale alone. By training on heterogeneous instruction–response pairs, the model internalizes a latent task distribution that allows it to interpret novel instructions without task-specific fine-tuning. This result established instruction fine-tuning as a distinct and powerful paradigm, forming the conceptual foundation for subsequent work in alignment and domain adaptation.

2.2 InstructGPT: Instruction Following and Alignment with Human Feedback

While FLAN established the effectiveness of instruction fine-tuning for generalization, it did not directly address issues of safety, helpfulness, or alignment with human preferences. These concerns were tackled by Ouyang et al. in *Training Language Models to Follow Instructions with Human Feedback* (InstructGPT) [52].

InstructGPT introduced a three-stage training pipeline that remains the basis for many production LLMs:

1. **Supervised instruction fine-tuning (SFT)** on human-written instruction–response pairs;
2. **Preference modeling** using human rankings of multiple model outputs;
3. **Reinforcement learning from human feedback (RLHF)** to optimize model behavior toward preferred responses.

A key result of this work was that relatively small models, when instruction-tuned and aligned via RLHF, were preferred by human evaluators over much larger pretrained models. This demonstrated that alignment and instruction-following quality can outweigh raw model scale.

From a historical perspective, InstructGPT complements FLAN by transforming instruction fine-tuning from a purely performance-driven technique into a broader alignment framework. Together, FLAN and InstructGPT established instruction fine-tuning as both a generalization mechanism and a control interface between humans and large language models.

2.3 FLAN-T5: Scaling Instruction Fine-Tuning

The third pillar of instruction fine-tuning was established by Chung et al. in *Scaling Instruction-Finetuned Language Models* (FLAN-T5) [16]. This work systematically explored how instruction fine-tuning behaves across model sizes, task mixtures, and dataset scales.

FLAN-T5 expanded the instruction corpus to thousands of tasks and demonstrated that instruction fine-tuning consistently improves performance across a wide range of downstream benchmarks, including reasoning and knowledge-intensive tasks. Importantly, the authors showed that instruction fine-tuning benefits models of all sizes, from small to very large, and that gains persist even as base models scale.

This result established instruction fine-tuning as a *scalable and robust training paradigm*, rather than a technique limited to a narrow regime. It also reinforced the idea that the diversity and structure of instruction data are as important as parameter count—a conclusion that directly motivates modern domain-specialized instruction datasets.

2.4 Implications for Domain-Specialized Instruction Tuning

Together, FLAN, InstructGPT, and FLAN-T5 define the historical and conceptual foundations of instruction fine-tuning:

- FLAN establishes that instruction-following can be learned and generalized;
- InstructGPT integrates instruction fine-tuning with human alignment objectives;
- FLAN-T5 demonstrates that instruction fine-tuning scales across model sizes and task mixtures.

Recent domain-specific efforts—such as CyberPal.AI and SecKnowledge [42, 43]—extend these principles by grounding instruction data in structured cybersecurity knowledge bases. The Athena CTI instruction framework further advances this trajectory by combining symbolic, graph-based instruction templates with benchmark-driven iteration via AthenaBench. In this sense, modern CTI-focused instruction fine-tuning can be viewed as a third-generation evolution of the original FLAN paradigm: instruction learning grounded not only in natural language, but in explicit domain structure and formal knowledge representations.

3 Instruction Fine-Tuning and Small Language Models

3.1 From Pretraining to Instruction-Following

Pretrained LLMs are optimized for next-token prediction on broad text corpora, not for following explicit user instructions. Instruction tuning—also called instruction fine-tuning (IFT)—fine-tunes such models on labeled datasets of instructions and target outputs, improving their ability to interpret prompts as tasks rather than mere text prefixes.[84, 52]

Key characteristics of instruction tuning include:

- **Data format.** Each sample consists of an instruction, optional context, and a desired output.
- **Supervised objective.** The model is trained to maximize log-likelihood of the reference output given the instruction and context, often combining a standard language modeling loss with an instruction-following loss.
- **Generalization across tasks.** Instruction datasets deliberately mix many task types (QA, classification, summarization, transformation) so that the fine-tuned model can generalize to novel instructions, as shown by InstructGPT and FLAN.[84, 52]

IFT is now routine in the model lifecycle: base models (e.g., LLaMA-like, Granite, Qwen) are first pretrained, then instruction-tuned, and often further refined via reinforcement learning from human feedback (RLHF) and safety fine-tunes.[52]

3.2 Instruction Fine-Tuning for Cybersecurity

Cybersecurity introduces domain-specific challenges:

- Technical jargon and heterogeneous schemas (CVE, CWE, CAPEC, ATT&CK).
- Need for structured, evidence-grounded outputs (e.g., CVSS vectors, CWE IDs, ATT&CK mappings).
- Rapid concept drift as new vulnerabilities and campaigns appear.

Several efforts address this via cyber-specific instruction data:

- **SEvenLLM.** Ji et al. construct a bilingual cybersecurity instruction dataset and a CTI benchmark (SEvenLLM-Bench) from real incident reports, converting raw text into supervised QA, extraction, and reasoning tasks across many task types.[34]
- **CyberBench / CyberInstruct.** Liu et al. create CyberInstruct, a set of cyber-specific instruction tasks drawn from logs, alerts, and vulnerability descriptions, used to train models evaluated on CyberBench.[46]
- **Lily-Cybersecurity-7B.** Lily is a Mistral-7B fine-tune on hand-crafted cybersecurity and hacking instructions, later enriched by an LLM for style and consistency.[46]

The **CyberPal.AI** line of work from IBM pushes this further by tightly integrating CTI knowledge bases and detection artifacts into an expert-driven instruction pipeline:

- **SecKnowledge (CyberPal 1.0).** Levi et al. build an instruction dataset by mining MITRE ATT&CK, Sigma rules, SIEM rules, adversary emulation plans, and threat reports, then using domain experts to define instruction schemas (e.g., mapping logs to ATT&CK techniques, suggesting mitigations) and populate them semi-automatically.[42]
- **IFT results.** CyberPal models instruction-tuned on SecKnowledge (based on LLaMA-like and other open models) outperform base models by up to roughly 24% on security tasks and show improved robustness on CTI benchmarks.[42]

The follow-on **"Toward Cybersecurity-Expert Small Language Models"** paper extends this to SecKnowledge 2.0, enriching instructions with chain-of-thought (CoT) rationales and designing a pipeline with expert-in-the-loop validation and LLM-guided knowledge retrieval, then training cybersecurity-expert SLMs (4B–20B parameter models) that rival or surpass much larger general models on CTI tasks.[43]

These works demonstrate that expert-designed, knowledge-grounded instruction datasets can transform general LLMs into specialized CTI assistants with strong reasoning and reduced hallucination—precisely the niche targeted by the Athena template-based IFT system.

3.3 Small Language Models for Cybersecurity

SLMs—roughly 2–20B parameters—are increasingly attractive for cybersecurity:

- **Deployment constraints.** Many SOC and CTI environments require on-premises, air-gapped, or "bring-your-own-model" setups for regulatory and confidentiality reasons.
- **Latency and cost.** Incident response and detection engineering often require low-latency responses and large volumes of queries; SLMs are simpler to host and scale.
- **Specialization.** With strong domain IFT, SLMs can match or exceed larger general models on specialized tasks while being easier to retrain as CTI sources evolve.[43]

CyberPal 2.0 explicitly targets this: it compares cybersecurity-expert SLMs (4B–20B) against both larger open models and proprietary systems across CTI knowledge tasks, finding that well-instruction-tuned SLMs can achieve competitive or superior performance on several domains, especially when trained with CoT-enriched instructions and CTI-aware formatting.[43]

The emerging picture is that *IFT + SLM + domain knowledge* is a pragmatic recipe for CTI: the heavy lifting (linguistic knowledge) comes from pretraining; domain semantics and task formats come from IFT; runtime and deployment practicality come from SLM size. The Athena IFT dataset generation system is designed to feed this recipe by generating large quantities of high-fidelity instruction triples directly from CTI knowledge bases.[13]

4 Reinforcement Learning: Historical Developments and Current State

Reinforcement Learning (RL) is the study of how agents learn to select actions through interaction with an environment so as to maximize cumulative reward. Although the modern field is often associated with machine learning and deep neural networks, its intellectual roots span multiple disciplines: experimental psychology (trial-and-error learning), operations research (stochastic control), and computer science (adaptive algorithms). This section summarizes major research milestones in RL, emphasizing foundational concepts, canonical algorithmic breakthroughs, and key inflection points that shaped the modern landscape.

4.1 Origins: Trial-and-Error Learning and Early Decision Theory

One intellectual root of RL lies in early psychological theories of learning by consequences. Thorndike's *Law of Effect* formalized the idea that behaviors followed by satisfying outcomes become more likely, providing a conceptual precursor to reward-driven adaptation.[78] Skinner's operant conditioning further advanced this framework by emphasizing reinforcement schedules and behavioral shaping, ideas that later influenced computational notions of reward and feedback.[67]

In parallel, statistical decision theory developed formal treatments of sequential decision-making under uncertainty. A foundational example is the multi-armed bandit problem, introduced in a modern form by Robbins.[57] Bandits later became a central sub-area of RL and online learning, with influential developments such as Thompson sampling,[77] the Gittins index,[24] and regret-minimizing algorithms such as UCB.[8]

4.2 Formal Foundations: Dynamic Programming and Markov Decision Processes

The mathematical foundations of RL crystallized through dynamic programming (DP) and Markov decision processes (MDPs). Bellman introduced DP and the principle of optimality, providing a general method for computing optimal value functions through recursive decomposition.[11] Howard's monograph systematized MDPs and policy iteration, establishing a formal basis for optimal sequential decision-making in stochastic environments.[31]

These works framed the core objects that remain central today: states, actions, transition dynamics, reward functions, value functions, and optimal policies. They also clarified the algorithmic blueprint for many RL methods: estimate (or learn) value functions and improve policies iteratively (policy iteration), or directly compute optimal value functions (value iteration).[11, 31]

4.3 Computational Reinforcement Learning: TD Learning, Actor–Critic, and Q-Learning

The modern computational RL era emerged as researchers sought methods that could learn from experience without a known model of the environment. A key breakthrough was temporal-difference (TD) learning, introduced by Sutton, which combined ideas from Monte Carlo evaluation and DP bootstrapping.[73] TD learning enabled agents to learn value functions online from partial episodes and noisy experience, becoming a cornerstone of RL.

Around the same period, actor–critic architectures appeared as an approach to combine policy learning (the *actor*) with value estimation (the *critic*). Barto, Sutton, and Anderson showed that "neuronlike adaptive elements" could solve learning control problems via such interacting components, foreshadowing modern policy-gradient actor–critic methods.[9]

Another landmark was Q-learning, introduced by Watkins (and later formalized with Dayan), which provided an off-policy TD control method capable of learning optimal action-values without a model and without requiring

on-policy sampling.[81, 82] This was particularly influential because it separated learning about the optimal policy from the behavior policy used to explore, enabling flexible exploration strategies.

4.4 Function Approximation and "Approximate Dynamic Programming"

As RL moved beyond small tabular problems, function approximation became essential. Least-squares and approximation methods improved sample efficiency and stability for value estimation, including least-squares temporal difference learning (LSTD).[12] Policy iteration combined with function approximation led to influential algorithms such as Least-Squares Policy Iteration (LSPI), which helped bridge RL and supervised regression methods.[39] Batch RL and fitted methods (e.g., fitted Q iteration) further advanced the use of regression trees and other approximators for value learning.[20]

Policy-gradient methods developed as a complementary direction, directly optimizing parameterized policies. Williams' REINFORCE provided a widely cited policy-gradient estimator.[87] Later work clarified policy-gradient methods with function approximation and established theoretical foundations for actor–critic algorithms.[76, 37]

4.5 Model-Based RL and Planning: From Dyna to Modern Learned World Models

Model-based RL explores how learning a model of environment dynamics can reduce sample complexity by enabling planning and simulated experience. Sutton's Dyna architecture unified learning, planning, and acting: agents learn a model, generate simulated transitions, and update value/policy estimates using both real and imagined data.[74] This integration of learning and planning remains a recurring theme, especially in robotics and safety-critical settings.

In continuous control and robotics, model-based policy search methods such as PILCO highlighted the potential for very data-efficient learning by combining probabilistic dynamics models with policy optimization.[19] More recent work revisits model-based RL at scale using neural latent dynamics ("world models") and imagination-based rollouts.[27, 33]

4.6 The Deep RL Inflection Point: DQN and Beyond

The modern deep RL era is often traced to Deep Q-Networks (DQN), which combined Q-learning with deep convolutional networks and introduced key stabilizing mechanisms such as experience replay and target networks.[49] DQN's Atari results demonstrated that a single architecture could learn control policies from high-dimensional sensory inputs across many tasks, catalyzing broad interest in RL as a general-purpose learning framework.

Subsequent work addressed known instabilities and improved performance through algorithmic refinements such as Double DQN,[79] prioritized experience replay,[58] distributional RL,[10] and integrated combinations (e.g., Rainbow).[30]

In parallel, policy-gradient methods became dominant for continuous control and high-dimensional action spaces. Trust Region Policy Optimization (TRPO) and Proximal Policy Optimization (PPO) became widely used due to favorable stability/performance tradeoffs.[60, 61] Actor–critic methods such as A3C and deterministic policy gradients (DDPG) further expanded the practical toolkit.[48, 45] Entropy-regularized objectives (maximum-entropy RL) culminated in algorithms such as Soft Actor-Critic (SAC), which improved stability and exploration in continuous domains.[26]

4.7 Game-Playing Milestones: AlphaGo, AlphaZero, and MuZero

A major public milestone was AlphaGo, which combined deep neural networks with Monte Carlo tree search (MCTS) to defeat top human Go players.[65] AlphaZero simplified the approach by learning entirely from self-play without human expert data, demonstrating that general game-playing competence could emerge from a unified RL framework.[66] MuZero extended these ideas by learning a latent dynamics model sufficient for planning, enabling strong performance on Atari and board games without access to the true environment dynamics.[59]

These systems helped popularize several themes: scalable self-play, the synergy of planning and learning, and the value of strong inductive biases in model architecture and training regimes.

4.8 RL from Demonstrations, Inverse RL, and Preference-Based Feedback

Beyond reward engineering, several lines of work explore how to infer objectives or policies from behavior. Inverse RL (IRL) formalizes the problem of recovering a reward function consistent with expert demonstrations.[51] Apprenticeship learning and maximum-entropy IRL advanced practical IRL and structured probabilistic formulations.[4, 97]

Preference-based RL introduced a related paradigm: instead of demonstrations or explicit rewards, agents learn from human comparisons among trajectories.[15] This preference-learning thread later became central to post-training large language models via RLHF-style pipelines (supervised instruction tuning + preference models + RL optimization), a major milestone in aligning generative systems.[52]

4.9 Offline RL and Data-Centric Learning

As RL deployments expanded, offline (batch) RL gained prominence: learning policies from fixed logged datasets without further environment interaction. Offline RL is attractive in domains where exploration is expensive or unsafe, but it introduces distribution-shift and extrapolation challenges. Modern algorithms such as BCQ and CQL exemplify the renewed focus on conservative value estimation and policy constraints under dataset support limitations.[22, 38]

4.10 Current State and Open Challenges

Today, RL spans a spectrum from bandits and tabular MDPs to deep RL, model-based planning, multi-agent systems, and LLM post-training. Despite dramatic progress, several persistent challenges remain active research areas:

- **Sample efficiency and generalization:** many RL systems still require large interaction budgets and can generalize poorly outside training distributions.[75]
- **Stability with function approximation:** off-policy learning with nonlinear approximators can be unstable; practical success often relies on careful engineering and regularization.[49]
- **Exploration and credit assignment:** sparse rewards and long horizons remain difficult, motivating intrinsic motivation, planning, and structured objectives.[74]
- **Safety and constraints:** safe RL and constrained MDPs formalize safety requirements, but robustly enforcing them under uncertainty is challenging.[7, 23]
- **Evaluation methodology:** benchmarks can be sensitive to implementation details and environment stochasticity; reproducibility remains a concern.[28]

A unifying perspective is that RL has repeatedly advanced through the co-evolution of (i) formal objectives (MDPs, constraints), (ii) algorithmic innovations (TD, Q-learning, policy gradients, planning), and (iii) enabling infrastructure (simulation platforms, scalable compute, differentiable function approximation). This historical pattern continues as RL concepts increasingly influence the training and alignment of large generative models.

5 Reinforcement Learning with Verifiable Rewards (RLVR): History, Milestones, and Open Challenges

5.1 Definition and Motivation

Reinforcement Learning with Verifiable Rewards (RLVR) is an increasingly prominent post-training paradigm for large language models (LLMs) in which the reward signal is computed by an automated *verifier* rather than by a learned preference/reward model. In its most typical form, the verifier produces a binary signal (pass/fail) derived from *objective checks* such as: (i) exact-match or normalized-match correctness against a known gold answer, (ii) execution-based verification (e.g., unit tests for generated code), or (iii) satisfaction of explicit output constraints (formatting constraints, structured outputs, or refusal constraints) [40, 50].

RLVR is often positioned as a complementary alternative to reinforcement learning from human feedback (RLHF), which relies on human preference data and a learned reward model as an intermediate supervisory signal [53]. While RLHF has proven effective for instruction following and conversational helpfulness, learned reward models can be expensive to construct, difficult to generalize across tasks, and vulnerable to reward hacking and specification gaming [53]. RLVR addresses part of this problem by replacing the reward model with a verifier whose signal is (in principle) grounded in externally checkable outcomes [40, 50]. However, the promise of "objective" rewards does not remove all alignment challenges: many tasks lack unambiguous gold standards, verification can be sparse, and even constraint-based verifiers can be gamed when optimization pressure is high [50, 63].

5.2 Antecedents: RL for Text Generation and Program Synthesis

Although RLVR (as a named paradigm for LLM post-training) is recent, it builds on earlier work using policy-gradient reinforcement learning to optimize non-differentiable objectives in sequence generation. Early milestones include REINFORCE-style objectives for language generation [88] and sequence-level training that directly optimizes task metrics (e.g., BLEU/ROUGE) to reduce exposure bias [56, 54]. While these earlier approaches used automatically computed rewards, the rewards were typically *proxy metrics* rather than correctness verifiers, and they often suffered from high-variance gradients and instability.

A closer precursor to modern RLVR is reinforcement learning for code generation/program synthesis with *execution-based* feedback. Here, unit tests provide a naturally verifiable reward, enabling learning loops in which candidate programs are generated, executed, and scored by pass/fail outcomes. CodeRL is an influential example that integrates pretrained code LMs with RL signals derived from functional correctness and unit tests [41]. This "execute-to-verify" idea later becomes a central pillar of RLVR for both code and reasoning tasks [40, 50].

5.3 Reasoning Supervision and the Rise of Verifiers

The modern RLVR story is tightly coupled to the rapid growth of *explicit reasoning* in LLMs (e.g., chain-of-thought prompting) and the corresponding need for more reliable supervision signals [85]. Reasoning benchmarks such as MATH provide structured problems with checkable answers and step-by-step solutions, making them fertile ground for correctness verification and for studying different forms of supervision [29].

A major milestone in "verifier-centric" training is *process supervision*, which provides feedback at intermediate reasoning steps rather than only at the final answer. "Let's Verify Step by Step" demonstrates that step-level supervision can outperform outcome-only supervision on mathematical reasoning, and it released PRM800K, a large dataset of step-level labels for training process reward models [44]. While process supervision is not identical to RLVR (it often involves learned reward models), it substantially influenced the field's emphasis on *verifiers* and *credit assignment* in multi-step reasoning.

5.4 RLVR as a Named Paradigm: 2024–2025

RLVR became explicitly formalized and popularized in open LLM post-training work during 2024–2025. The Tulu 3 report is a key reference point: it introduces RLVR as an open post-training recipe alongside supervised fine-tuning and preference optimization, and it articulates classes of verifiable rewards (correctness checks, execution checks, and verifiable constraints) [40].

In parallel, DeepSeekMath introduced Group Relative Policy Optimization (GRPO), a PPO-variant designed to improve memory efficiency while optimizing verifiable rewards for mathematical reasoning [64]. DeepSeekMath is historically important because it helped establish GRPO-style training as a practical route to scaling RL on LLMs under sparse binary rewards.

Another important development is the push toward denser and more faithful intermediate feedback. "Rewarding Progress" scales automated process verifiers for reasoning, explicitly targeting the sparse-reward/credit-assignment bottleneck in multi-step problems [62]. Collectively, these works set the stage for RLVR to become one of the principal mechanisms behind "reasoning model" post-training.

5.5 RLVR at Scale for Reasoning Models

A watershed moment for RLVR was DeepSeek-R1, which demonstrated that large-scale reasoning capabilities can be substantially incentivized through reinforcement learning using rule-based/verifiable signals, without requiring human-labeled reasoning trajectories as the primary driver [18]. DeepSeek-R1 also helped consolidate several practical conventions used in RLVR training loops (e.g., structured reasoning traces, strong KL constraints, and verifier-driven pass/fail rewards on tasks like math and code) [18].

Following this wave, several efforts aimed to scale and operationalize RL training for LLMs more broadly. Kimi k1.5 describes scaling reinforcement learning with LLMs, contributing additional evidence that RL-style post-training can be productively scaled beyond narrow demos [36]. DAPO provides an open-source RL system at scale, with an emphasis on reproducible recipes and system-level engineering choices that make large-scale LLM RL more attainable for the research community [91]. FastCuRL explores curriculum and context-scaling strategies to make training "R1-like" reasoning models more efficient [68].

5.6 Theory, Optimization Stability, and Reward Shaping

As RLVR spread, the community rapidly confronted stability issues associated with sparse and discrete rewards. A key theoretical contribution is Mroueh's analysis of GRPO with verifiable rewards, interpreting GRPO as a KL-regularized contrastive loss and formally characterizing its "success amplification" dynamics under binary rewards [50]. This line of work is important because it begins to connect empirical RLVR gains with tractable mathematical models of optimization dynamics.

On the practical side, techniques for smoothing or reshaping discrete reward signals emerged. ReDit proposes "reward dithering" to mitigate gradient anomalies and convergence issues under discrete (0/1) reward functions [83]. More recently, VeRPO proposes verifiable *dense* rewards for code generation, explicitly addressing the credit-assignment problem when the only naturally verifiable signal is final functional correctness [80].

5.7 Debate: Does RLVR Create New Reasoning?

A central scientific question is whether RLVR elicits fundamentally new reasoning strategies or primarily improves *sampling efficiency* and calibration of behaviors already latent in the base model. Yue et al. argue that, under many current training setups, RLVR-trained models may outperform at small k (e.g., pass@1) while the base model can dominate at sufficiently large k (pass@k), suggesting RLVR may not reliably expand the model's intrinsic reasoning boundary [93].

In contrast, Wen et al. provide evidence that RLVR can extend reasoning boundaries for math and code, and they propose CoT-Pass@K to capture success that depends on intermediate reasoning quality rather than only final-answer correctness [86]. These complementary findings highlight that evaluation methodology and the choice of success criteria (final answer vs. process correctness) materially affect conclusions about what RLVR is learning.

Adding complexity, "Spurious Rewards" reports that improvements can occur even under counterintuitive or imperfect reward signals, challenging overly simple narratives about RLVR's mechanism and raising the possibility of shortcut learning or distributional effects [63]. Subsequent mechanistic work ("Spurious Rewards Paradox") studies how RLVR can activate memorization shortcuts, reinforcing the need to disentangle capability gains from dataset artifacts and optimization-induced shortcuts [90].

5.8 Beyond Strict Verifiability: Broad Domains and Open-ended Generation

A major limitation of classical RLVR is that many real tasks are not cleanly verifiable. "Crossing the Reward Bridge" investigates RLVR beyond math/code by leveraging expert reference answers and developing soft, model-based reward signals when binary verification is infeasible [71]. This work is historically significant because it articulates a practical pathway for extending RLVR to domains with ambiguity and partial correctness.

Domain-specialized RLVR also began to appear. Med-RLVR demonstrates emergent medical reasoning from a 3B base model using RL, illustrating that verifier-like objectives can be adapted to specialized professional domains when appropriate references and scoring are available [94]. For more open-ended generation, RLVRR proposes "verifiable reference-based reward chains" as a structured approach to using reference material to build more informative training signals than a single binary check [35].

5.9 Safety, Alignment Drift, and Verifiable Constraints

Another open question is how RLVR interacts with safety alignment. While RLVR is often assumed to be "safer" than reward-model-based RLHF (because the objective is externally checkable), the safety implications depend heavily on what is being verified and how the KL constraint is enforced. "Breaking the Safety-Capability Tradeoff" argues—both theoretically and empirically—that RLVR can improve reasoning while maintaining or even improving safety guardrails under certain KL-constrained regimes [14]. At the same time, the broader literature emphasizes that verifiable constraints can still induce reward hacking or undesirable behavior if the constraint is misspecified or optimization pressure is too strong [40, 50].

5.10 Current State and Open Challenges

Despite rapid progress, RLVR remains an active research frontier. Key challenges include:

- **Verifier design and task coverage.** Many domains lack ground-truth answers or executable checks; building reliable verifiers (or reference-based reward chains) without collapsing to model-as-judge bias is unresolved [71, 35].
- **Sparse rewards and credit assignment.** Binary rewards are information-poor for long-horizon reasoning; process verifiers and dense shaping (where feasible) are essential but can introduce new failure modes [44, 62, 80].
- **Stability and efficiency at scale.** Discrete rewards and high-variance updates motivate algorithmic refinements (GRPO analyses, reward dithering, curricula, and system-level scaling recipes) [50, 83, 68, 91].
- **Scientific understanding of what improves.** Competing results on whether RLVR expands reasoning vs. reweights sampling underscore the need for stronger evaluations and mechanistic interpretability [93, 86, 90].
- **Safety and alignment drift.** RLVR may mitigate some RLHF weaknesses, but it is not automatically aligned; safety depends on the verifier, the KL constraint, and the broader training recipe [14, 40].

In summary, RLVR has evolved from early RL-for-text and execution-verified code training into a central paradigm for modern reasoning-oriented LLM post-training. Its future trajectory will likely depend on advances in verifier construction, process-level supervision, reward shaping, and rigorous evaluation methodologies that can distinguish genuine reasoning improvements from shortcut learning.

6 Reinforcement Learning with Verifiable Rewards (RLVR)

Reinforcement Learning with Verifiable Rewards (RLVR) is an increasingly prominent post-training paradigm for large language models (LLMs) in which model outputs are optimized using reinforcement learning (RL) against rewards computed by an *external verifier* rather than a learned reward model or direct human preference comparisons. In RLVR, correctness is *objectively checkable* (or at least operationally checkable) via deterministic or programmatic criteria such as exact-match answers, unit-test execution, compilation, constrained formatting, or other machine-checkable validations [25, 17, 86].

RLVR has become especially salient for *reasoning-style language generation*—where the model produces a multi-step natural language (or mixed natural language + formal) solution—because many reasoning tasks admit reliable verifiers (e.g., math final answers, code execution, multiple-choice ground truth) [25, 86, 94]. At the same time, RLVR raises distinctive research questions around reward sparsity, exploration, generalization, and evaluation (e.g., when RLVR improves *sampling efficiency* versus *reasoning capacity*) [86, 50].

6.1 Motivation: From RLHF to RLVR

Much of modern LLM post-training can be viewed as choosing (i) a reward signal and (ii) an optimization procedure to shift a pretrained model toward desirable outputs. A canonical predecessor is RL from Human Feedback (RLHF), where a reward model is trained on human preference comparisons and then used to optimize the policy (the LLM) with an RL algorithm such as PPO [15, 98, 69, 52, 61]. RLHF is powerful for open-ended language quality and helpfulness, but it is costly (human labels), can be brittle (reward model misspecification), and is susceptible to reward hacking (policy exploits reward model artifacts).

RLVR shifts the supervision burden from preference data to *verification infrastructure*. When a task admits a robust automatic checker, RLVR can (a) reduce reliance on large-scale human labeling, (b) mitigate some forms of reward hacking by eliminating learned reward models, and (c) encourage exploration over solution strategies while still grounding optimization in correctness [25, 86, 14]. However, RLVR is constrained by verifier availability and by the statistical and optimization challenges induced by sparse, discrete reward signals [83, 80].

6.2 What Counts as a "Verifiable" Reward?

In the RLVR literature, *verifiable* typically means there exists an automated procedure that maps a model output to a reward with high reliability and low ambiguity. Common verifier types include:

- **Exact-match / normalized-match checkers.** Many math and QA datasets support rewards based on string/number matching after normalization (e.g., canonical numeric forms, symbolic simplification).

- **Execution-based checkers for code.** Rewards are derived by compiling/running generated code against unit tests (binary pass/fail or richer execution traces) [41, 80].
- **Closed-form structured ground truth (e.g., MCQA).** Multiple-choice tasks provide verifiable correctness labels and are frequently used as a "bridge" domain for RLVR beyond math/code [94].
- **Constraint and schema validation.** Format compliance (valid JSON, regex patterns, type constraints, tool-call schemas) can provide partial, verifiable rewards; this is particularly relevant in agentic and tool-using LLM settings.

A practical nuance is that many "real" language tasks are only partially verifiable. Recent work explores hybrids that combine *binary verification* where possible with softer scoring or model-based grading where ground truth is less structured, effectively expanding RLVR-style training to more domains [72].

6.3 RLVR Formulation for Autoregressive Language Models

RLVR treats an autoregressive LLM as a stochastic policy π_θ that generates an output sequence $y = (y_1, \ldots, y_T)$ conditioned on an input prompt x. A verifier computes a scalar reward $r(x, y)$, often only after the full sequence is generated (delayed reward), though variants incorporate intermediate or decomposed rewards when partial verification is feasible [80].

A common post-training objective mirrors KL-regularized RLHF formulations, but with a verifiable reward:

$$\max_{\theta} \; \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)} \left[r(x,y)\right] \; - \; \beta \, \mathbb{E}_{x\sim\mathcal{D}} \left[\mathrm{KL}(\pi_\theta(\cdot \mid x) \,\|\, \pi_{\mathrm{ref}}(\cdot \mid x))\right], \tag{1}$$

where π_{ref} is a reference policy (often the pre-RL model) and β controls the update magnitude. KL regularization is used to stabilize optimization and limit destructive drift, and it is also central to safety analyses of RLVR-style post-training [14].

6.4 Optimization Algorithms: PPO, GRPO, and Group-Based RL

Early work applying RL to sequence generation relied on policy-gradient estimators such as REINFORCE [87] and introduced sequence-level RL objectives to address exposure bias and metric mismatch (e.g., optimizing BLEU/ROUGE directly) [56, 55]. In modern LLM post-training, PPO became a widely used workhorse due to its practical stability [61, 52].

A key recent development in RLVR is *group-based* policy optimization, especially Group Relative Policy Optimization (GRPO), introduced in the DeepSeekMath line and used prominently in DeepSeek-R1-style reasoning training [17, 25]. Instead of learning a separate value function baseline, GRPO samples a *group* of K completions per prompt, computes rewards for each completion, and forms an advantage estimate via within-group normalization. This reduces memory overhead (no value network) and can scale well in large post-training regimes [17, 50].

Recent analyses connect GRPO-style updates to contrastive objectives and study how reward normalization (mean-only vs. mean+variance) and KL regularization choices influence convergence and "success amplification" [50]. At the same time, empirical work highlights an important evaluation subtlety: improvements measured by Pass@1 may mask tradeoffs in Pass@K and reasoning diversity; refined metrics such as CoT-Pass@K have been proposed to better capture reasoning correctness beyond final-answer matching [86].

6.5 Major Research Milestones and Timeline

From a research-history perspective, RLVR for language generation builds on several threads:

- **Policy gradients and RL for sequence generation (1990s–2010s).** REINFORCE provided foundational estimators [87]. Sequence-level RL objectives for text generation were explored to address exposure bias and optimize non-differentiable metrics [56, 55].
- **RL from preferences → RLHF for language (2017–2022).** Preference-based reward learning [15] was adapted to language tasks [98, 69] and then scaled into instruction-following RLHF pipelines (e.g., InstructGPT) [52].

- **Execution/verifier feedback for code generation (2022–).** Systems such as CodeRL operationalized verifiable, execution-based signals (unit tests) in RL fine-tuning for code generation [41].
- **Modern RLVR for reasoning and scalable group-based RL (2024–2025).** DeepSeekMath introduced GRPO in the context of math reasoning [17]. DeepSeek-R1 popularized large-scale RLVR-style reasoning post-training and catalyzed a wave of replication/analysis work [25, 86, 50].
- **Expanding RLVR beyond "clean" verifiable domains (2025–).** Work such as "Crossing the Reward Bridge" studies extensions to domains where reference answers are less structured and explores hybrid verification + soft scoring approaches [72]. Domain studies (e.g., medical MCQA) show RLVR-style training can generalize beyond math/code under verifiable labels [94].
- **Safety, stability, and reward-design advances (2025–2026).** Safety analyses study how KL-constrained RLVR can maintain safety guardrails under certain conditions [14]. Methods such as reward dithering address optimization instabilities from discrete rewards [83], and verifiable dense reward methods (e.g., weighted partial test success) aim to reduce reward sparsity without introducing learned reward models [80].

6.6 Current State and Open Challenges

RLVR is now a central technique for producing strong reasoning-oriented LLMs in settings where correctness can be checked automatically [25, 86]. However, several challenges remain open and are active areas of research:

Reward sparsity, variance, and credit assignment. Many RLVR deployments rely on *binary* outcome rewards (correct/incorrect), which can yield high-variance gradient estimates and stall training when sampled groups receive uniform rewards. Techniques such as reward perturbation (dithering) [83] and verifiable *dense* reward construction from partial success [80] aim to provide smoother learning signals while preserving objective grounding.

Verifier reliability, brittleness, and exploitability. A "verifiable" reward is only as good as the verifier. Execution-based rewards can be brittle (timeouts, nondeterminism, hidden tests), and format/constraint rewards can be gamed (the model optimizes superficial structure). Even with deterministic checkers, policies can exploit gaps between training-time verification and deployment-time desiderata (a form of reward hacking at the verifier boundary).

Generalization beyond verifiable domains. A core limitation of RLVR is that many real-world language tasks (summarization quality, helpfulness, harmlessness, nuanced correctness) are not fully verifiable. Recent research explores bridging strategies: using verifiable sub-tasks, combining verifiers with soft scoring, and leveraging cross-model agreement to approximate verification when ground truth is partially structured [72].

Evaluation: "sampling efficiency" vs. "reasoning capacity." A recurring debate is whether RLVR creates genuinely new reasoning capabilities or primarily reallocates probability mass toward already-present solution trajectories. Empirical work suggests that naive Pass@K can be misleading when base models can "guess" correct short answers without coherent reasoning, motivating metrics that verify intermediate reasoning quality (e.g., CoT-Pass@K) [86].

Safety and alignment under RLVR. RLVR can induce behavioral drift because it directly optimizes for task success. Recent work studies conditions (notably KL constraints and training methodology choices) under which RLVR can improve capability without degrading safety guardrails, but this remains a delicate and application-dependent design problem [14].

Systems cost and scaling. RLVR is inherently *sampling-intensive*: it requires generating many trajectories per prompt, running verifiers (often expensive execution), and performing on-policy updates. As a result, practical RLVR pipelines must balance rollout budget, verifier cost, stability constraints (KL), and performance gains—and may benefit from group-based methods (e.g., GRPO) that reduce memory overhead [17, 50].

6.7 Practical Takeaways for Language-Generation RLVR Pipelines

From an engineering and research-design standpoint, RLVR for language generation tends to work best when: (i) the task can be cast into a format with high-fidelity verification, (ii) reward sparsity is mitigated via group sampling, partial credit, or carefully designed dense-but-verifiable reward signals, (iii) KL regularization and evaluation protocols are chosen to avoid overfitting to the verifier and to preserve general language competence, and (iv) benchmarking explicitly distinguishes final-answer correctness from reasoning quality when the application requires faithful multi-step generation [86, 80, 14].

7 Cyber Threat Intelligence and Knowledge Graphs

7.1 CTI Concepts and Workflows

Cyber Threat Intelligence (CTI) is commonly defined as the collection, analysis, and dissemination of information about threats and adversaries to inform defensive decisions.[47] CTI workflows include:

- Ingesting unstructured reports (vendor blogs, whitepapers, advisories).
- Structuring them into entities and relations: vulnerabilities (CVEs), weaknesses (CWEs), attack patterns (CAPEC, ATT&CK), threat actors, campaigns, malware, tools, TTPs.
- Reasoning over this structure to answer questions such as:
  - Which techniques does this malware implement?
  - Which mitigations cover these techniques on a given platform?
  - Which threat actor is most likely behind this cluster of activity?
  - How severe and exploitable is a new CVE in our environment?

CTI sharing and CTI knowledge graphs emphasize the importance of standardized schemas (e.g., STIX/TAXII), community knowledge bases (MITRE ATT&CK, CAPEC, CWE), and APIs like NVD for building structured representations that can be consumed by automation.[70, 2, 3, 1]

7.2 MITRE Ecosystem and Related Schemas

The MITRE ecosystem underpins much of modern CTI:

- **MITRE ATT&CK.** A globally accessible knowledge base of adversary tactics, techniques, and sub-techniques, originally released in 2013 and formally documented in "MITRE ATT&CK: Design and Philosophy".[70] ATT&CK represents adversary behavior as a graph where attack-pattern nodes (techniques) relate to software, groups (threat actors), data sources, and mitigations.
- **CVE and NVD.** The Common Vulnerabilities and Exposures system and NIST's National Vulnerability Database provide machine-readable vulnerability records with descriptions, affected products, CVSS metrics, and links to related CWE and CAPEC entries.[2]
- **CWE.** Common Weakness Enumeration captures abstract classes of software/hardware weaknesses, often linked from CVEs and CAPEC attack patterns.[3]
- **CAPEC.** A catalog of attack patterns describing how adversaries exploit weaknesses and how those attacks can be mitigated, with explicit links to CWE and often implicitly to ATT&CK techniques.[1]
- **MITRE ENGAGE.** A framework for adversary engagement strategies, which can be integrated into CTI graphs as a defensive complement to ATT&CK.[13]

The IFT design document explicitly models these sources and their relationships in the ASG Neo4j CTI graph, extracting node types, properties, and cross-schema edges (CVE→CWE, CVE→CAPEC, CAPEC→CWE, ATT&CK techniques→CAPEC, etc.).[13]

7.3 Athena's CTI Graph and Template Language

The *Instruction Fine Tuning Dataset Design* document by Cardei defines an IFT Dataset Generation System built around an ASG CTI graph database and a rich template language.[13] Key features include:

- **CTI source support.** The system supports three categories of CTI documents: (1) structural/graph sources (MITRE ATT&CK, CVE, CAPEC, CWE, MITRE ENGAGE, EPSS, CISA KEV); (2) LLM-assisted sources (SIGMA rules, MITRE CAR, Wazuh rules); and (3) proprietary sources (Flashpoint Ignite).
- **IFT triple format.** Each generated data point is an (instruction, question, answer) triple: the instruction specifies the role and style (e.g., "You are a CTI expert that gives precise and concise answers"), the question is the user-facing CTI query, and the answer is the ground-truth response derived from the CTI graph.
- **Template language.** Templates provide a high-level, declarative way to bind CTI graph nodes and produce triples. They support aliases for CTI types; attribute access through nested properties; filters (using equality and inequality) applied to list properties with any/all semantics; local variables bound to nodes; random selection from lists; list-comprehension-like constructs to expand relationships into lists in the output; invisible sections that define bindings and constraints without appearing in text; and source disambiguation via a `source#type` prefix (e.g., `attck#attack-pattern` versus `capec#attack-pattern`).[13]
- **Positive and negative examples.** The template language explicitly supports negative constraints (e.g., mitigations that do *not* mitigate this technique or data components that do *not* detect a technique), enabling the generation of hard negatives that teach models what does *not* follow from the CTI graph.
- **Cross-source reasoning templates.** Example templates mimic complex reasoning chains, such as: given a CVE description, identify the corresponding CWE weakness, then find a CAPEC attack pattern with high likelihood of attack using that weakness, then find an ATT&CK technique implementing that attack pattern.
- **Incremental generation.** The generator allows version and date filters at both document and concept level. Users can specify version ranges and last-modified intervals per CTI source; relationships are only used if at least one endpoint and the relationship fall within those windows.
- **Generation control and reporting.** A JSON job configuration file specifies CTI version ranges, template texts, per-template count limits and coverage fractions, output directories, and dataset versioning. The generation API produces both per-template triple files and a global report summarizing counts, coverage, and errors, which can be post-processed into human-readable formats.[13]

Conceptually, this system is a symbolic counterpart to the LLM-assisted instruction pipelines used in CyberPal and SEvenLLM: where CyberPal uses LLMs and experts to synthesize security instructions from unstructured sources, the Athena system uses a declarative template language over a CTI graph to generate strongly grounded and precisely controllable instruction triples.

7.4 CTI Knowledge Graphs and LLMs

Recent work shows that LLMs and CTI knowledge graphs are synergistic. Various efforts use LLMs to extract triples from CTI text and populate knowledge graphs, then perform link prediction and reasoning over those graphs to produce actionable recommendations.[21, 95, 89, 92, 96] Other systems build CTI-KGs directly from ATT&CK, CVE, threat reports, and open sources, then query them using LLMs or hybrid systems for tasks such as threat hunting and reporting.

The Athena IFT design follows a complementary pattern: *knowledge graph* $\rightarrow$ *templates* $\rightarrow$ *IFT triples* $\rightarrow$ *SLM*, with AthenaBench serving as a *knowledge graph* $\rightarrow$ *evaluation tasks* pipeline. This alignment is central to the next section.

8 Cybersecurity LLM Benchmarks and AthenaBench

8.1 Why Cybersecurity-Specific Benchmarks?

General benchmarks like GLUE, MMLU, and BIG-Bench measure broad language understanding and world knowledge but are poorly aligned with security practitioners' needs. Key gaps include:

- Lack of security-specific schemas (CVE, CWE, ATT&CK).
- Absence of tasks that test applied reasoning (e.g., mapping logs to techniques, linking vulnerabilities to mitigations, attributing campaigns).
- Static datasets that quickly become outdated in a rapidly changing threat landscape.

Hence a wave of cybersecurity benchmarks has emerged:

- **CyberMetric and SecEval.** Benchmarks focusing on security multiple-choice knowledge across domains, some derived from certification-style questions.[46, 32]
- **CyberBench.** A multi-task benchmark for cybersecurity LLMs, including classification, QA, log and report analysis, accompanied by a CyberInstruct training corpus.[46]
- **CTIBench.** A NeurIPS 2024 benchmark for CTI that includes tasks like CTI knowledge questions, vulnerability assessment, and technique extraction based on curated CTI corpora.[6]
- **SEvenLLM-Bench.** A bilingual CTI benchmark derived from real incident reports, designed alongside the SEvenLLM-Instruct dataset.[34]

Collectively, these benchmarks shift focus from generic QA to security-aware reasoning, but most rely on static corpora, so their examples gradually lose relevance as new vulnerabilities and campaigns emerge. AthenaBench squarely targets this limitation.

8.2 AthenaBench: A Dynamic CTI Benchmark

AthenaBench is introduced as a dynamic benchmark suite for evaluating LLMs on realistic, knowledge-intensive CTI tasks and explicitly extends CTIBench.[5, 6] Its key innovations include:

- **Dynamic data construction.** Instead of static corpora, AthenaBench connects to live CTI data sources and APIs (MITRE ATT&CK, NVD, CISA advisories, threat reports) to continuously generate benchmark samples within configurable time windows. Tasks such as root cause mapping (CVE→CWE) and severity prediction (CVSS) are drawn from recent CVEs and weaknesses, mitigating training-set leakage and static knowledge effects.[5]
- **Six complementary tasks.** AthenaBench defines: CTI Knowledge Test (CKT), Root Cause Mapping (RCM), Vulnerability Severity Prediction (VSP), Threat Actor Attribution (TAA), Risk Mitigation Strategy (RMS), and Attack Technique Extraction (ATE). Together, they probe CTI knowledge, root cause reasoning, structured severity prediction, abductive attribution, defensive planning, and TTP extraction.[5]
- **Enhanced metrics and aggregated score.** AthenaBench improves task-specific metrics and defines a unified aggregated score, making it easier to compare LLMs' CTI reasoning capabilities along different dimensions.[5]
- **Model evaluation.** The authors evaluate 12 LLMs, including GPT-4, GPT-4o, GPT-5, Gemini-2.5 Flash/Pro, Qwen variants, Llama 3/3.1/3.3, and a Llama-Primus merged model. Proprietary models (GPT-5, Gemini-2.5 Pro) lead overall, but all models struggle on reasoning-intensive tasks such as TAA and RMS, especially open-source ones.[5]

By using live CTI sources and emphasizing reasoning over current threats, AthenaBench moves CTI benchmarking closer to real-world analyst workloads and provides a stringent target for cybersecurity-expert SLMs.

8.3 Relationship to Athena's IFT Dataset Generator

AthenaBench and the Athena IFT generator are complementary components of a full CTI LLM lifecycle:

- **Training side (IFT generator).** Uses CTI graph data (ATT&CK, CVE, CAPEC, CWE, ENGAGE, etc.) and a declarative template language to generate instruction triples that drill CTI knowledge, multi-hop reasoning, negative cases, and structured outputs.[13]

- **Evaluation side (AthenaBench).** Dynamically constructs CTI tasks from the same or similar underlying sources (ATT&CK, NVD, CISA, threat reports) but with different pipelines and sampling strategies, providing independent, evolving test sets to evaluate model generalization.[5]

Many of the template patterns in the design document directly mirror AthenaBench tasks:

- CVE→CWE reasoning templates align with RCM.
- Templates that traverse CVE→CWE→CAPEC→ATT&CK map onto knowledge and mitigation reasoning tested in CKT and RMS.
- Templates involving malware, techniques, platforms, and data components parallel the kind of "who uses what and how is it detected" reasoning central to CTI analysis and attack technique extraction tasks.[13]

The incremental generation features of the IFT system (version and timestamp filtering) make it possible to align the training data's recency with AthenaBench's dynamic evaluation horizon, mitigating data leakage while ensuring models are trained on approximately the same vintage of CTI as they are evaluated on.[13, 5]

This alignment positions the Athena IFT generator as a natural training counterpart to AthenaBench's evaluation pipeline, much as SecKnowledge and CyberPal models are paired with CTI benchmarks in IBM's work.[42, 43]

9 Positioning Template-Based IFT for Cybersecurity SLMs

9.1 Design Goals and Advantages

Compared to purely LLM-generated instruction datasets, a template-driven, knowledge-grounded approach offers several advantages:

- **Ground-truth faithfulness.** Answers are computed directly from an authoritative CTI graph, reducing hallucinations and inconsistencies in training labels.
- **Explicit coverage control.** Templates can be designed to cover specific combinations of entities (e.g., CAPEC patterns affecting memory resources and their detection methods) or relationships (e.g., malware using two techniques on Linux), and per-template coverage fractions and count limits control dataset size and diversity.[13]
- **Rich negative examples.** Inequality constraints and relationship filters enable systematic generation of negative examples such as mitigations that do not address a technique or data components that do not detect any technique used by a malware family.
- **Explainability of templates.** Each template corresponds to a human-readable CTI reasoning pattern, making it easier for security experts to audit and refine the dataset.
- **Incremental refresh.** Version and date filters allow periodic regeneration of updated IFT corpora as CTI sources evolve, paralleling AthenaBench's dynamic evaluation but on the training side.[13, 5]

In combination with SLMs, this supports a training loop where:

- New CTI sources or schema updates lead to a refreshed CTI graph.
- CTI experts design or revise templates for new patterns and tasks.
- The generator produces updated IFT triples for fine-tuning SLMs.
- Models are evaluated on AthenaBench and other benchmarks.
- Discrepancies and weaknesses inform template revisions and possibly benchmark extensions.

9.2 Comparison with IBM's CyberPal and Other Instruction Pipelines

The Athena approach is conceptually closely related to SecKnowledge/CyberPal, but with different emphasis:

- **CyberPal / SecKnowledge.** Use expert-defined schemas and a pipeline combining LLMs and scripts to synthesize instructions from CTI sources and detection artifacts (e.g., Sigma rules). Emphasize human-curated "expert instruction patterns" and LLM-assisted generation, including CoT answers in SecKnowledge 2.0.[42, 43]
- **Athena IFT generator.** Uses a symbolic template language compiled into CTI graph queries, producing triples that are guaranteed to be consistent with the knowledge graph, with strong use of cross-schema constraints and negative examples, and built around incremental, configuration-driven generation.[13]

Both approaches are complementary:

- CyberPal demonstrates the effectiveness of expert instruction schemas and CoT enrichment in achieving high cybersecurity performance, especially in SLMs.
- Athena's system provides precise and reproducible control over content generation aligned with CTI schemas and knowledge graph structure, potentially feeding into similar CoT-enriched pipelines (e.g., by later adding CoT rationales using LLMs on top of graph-derived answers).

9.3 Toward Cybersecurity-Expert Small Language Models

Given these ingredients, an end-to-end cybersecurity-expert SLM pipeline could look like:

1. **CTI graph construction.** Continuously ingest and normalize CTI sources (ATT&CK, CVE, CAPEC, CWE, ENGAGE, ICS/OT sources, proprietary feeds) into a Neo4j or similar graph.
2. **Template authoring.** CTI experts encode important reasoning patterns (knowledge recall, root cause mapping, severity estimation, attribution clues, mitigation reasoning) as templates in the Athena language.
3. **IFT generation.** Run the generation tool with version and date constraints aligned to a training window, producing large, structured IFT datasets per task family.[13]
4. **Model training.** Fine-tune SLMs (e.g., 4–14B open models) using these IFT datasets, optionally adding CoT rationales, conversation styles, and alignment methods similar to CyberPal SecKnowledge 2.0.[43]
5. **Evaluation.** Benchmark models on AthenaBench and complementary suites (CTIBench, SecEval, CyberBench, SEvenLLM-Bench) to assess generalization and compare with larger proprietary models.[6, 5, 32, 46, 34]
6. **Feedback and iteration.** Use benchmark error patterns to design new templates, adjust CTI graph coverage, or refine IFT data (e.g., adding more negative cases or multi-hop chains).

This is closely aligned with IBM's vision of cybersecurity-expert SLMs but anchored in Athena's graph-driven IFT and dynamic CTI benchmarking.

10 Open Challenges and Research Directions

Despite rapid progress, several open questions remain:

- **Data leakage and benchmark freshness.** Dynamic benchmarks like AthenaBench reduce, but do not eliminate, the risk that training data (e.g., from the same CTI sources) overlaps heavily with evaluation samples. Careful time-based splits and provenance tracking between the IFT generator and benchmarks will be crucial.[5, 13]
- **Balancing symbolic templates and LLM creativity.** Template-based generation offers precision and coverage, but LLM-generated paraphrases can increase linguistic diversity and realism (e.g., multiple phrasings of the same CTI scenario). Hybrid pipelines that use templates to define semantics and LLMs to diversify surface form—while preserving correctness—are a promising area.

- **Incorporating chain-of-thought and explanations.** SecKnowledge 2.0 shows that CoT rationales significantly improve cyber reasoning.[43] Extending Athena templates to generate structured reasoning steps (via explicit graph traversal reasoning) and then augmenting them with LLM-generated natural language CoT could yield explainable CTI SLMs.
- **Operational evaluation dimensions.** AthenaBench focuses on knowledge and reasoning quality. Security operations also care about robustness to prompt injection, misuse, adversarial CTI, latency, and integration with SIEM/SOAR systems. Benchmarks like SecEval and CyberBench partly address this; extending AthenaBench and complementary suites to cover these aspects remains an important direction.[32, 46, 5]
- **Multilingual and cross-regional CTI.** SEvenLLM emphasizes bilingual corpora.[34] Current CTI benchmarks primarily use English sources, omitting intelligence from other languages that often contain region-specific threats. Template-based IFT over multilingual CTI graphs and multilingual benchmark tasks could help close this gap.
- **Human-in-the-loop and analyst trust.** For high-stakes CTI decisions, analysts need to understand why the model suggested a mapping or mitigation and how it connects to CTI sources. Template-driven data, CTI graph citations, and CoT rationales may enable verifiable, inspectable explanations.

11 Conclusion

Instruction fine-tuning and small language models are emerging as a practical and powerful combination for cybersecurity, especially in CTI workflows that demand structured reasoning, up-to-date knowledge, and on-premises deployment. IBM's CyberPal and SecKnowledge series demonstrate that expert-curated, CTI-grounded instruction datasets can turn SLMs into strong cybersecurity assistants, while AthenaBench raises the bar for CTI evaluation by integrating live CTI sources and focusing on reasoning-intensive tasks.[42, 43, 5]

The Athena IFT Dataset Generation System designed by Cardei and collaborators provides a complementary training-time infrastructure: a template language over a CTI knowledge graph that can generate rich, grounded, and incrementally refreshable instruction triples across multiple CTI schemas and tasks.[13] Together, these elements outline a coherent research program:

- Build and maintain high-fidelity CTI graphs from authoritative sources.
- Encode analyst reasoning patterns as templates that produce IFT data.
- Train cybersecurity-expert SLMs using these datasets.
- Evaluate them with dynamic CTI benchmarks like AthenaBench and iteratively refine both data and models.

The result is a principled pathway toward cybersecurity-expert small language models grounded simultaneously in CTI knowledge bases, instruction fine-tuning theory, and rigorous CTI benchmarking.

References

[1] Common attack pattern enumeration and classification (CAPEC). `https://capec.mitre.org/`. Accessed 2025-12-03.

[2] Common vulnerabilities and exposures (CVE). `https://www.cve.org/`. Accessed 2025-12-03.

[3] Common weakness enumeration (CWE). `https://cwe.mitre.org/`. Accessed 2025-12-03.

[4] Pieter Abbeel and Andrew Y. Ng. Apprenticeship learning via inverse reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2004.

[5] Md Tanvirul Alam, Dipkamal Bhusal, Salman Ahmad, Nidhi Rastogi, and Peter J. Worth. Athenabench: A dynamic benchmark for evaluating LLMs in cyber threat intelligence. *arXiv preprint arXiv:2511.01144*, 2025.

[6] Md Tanvirul Alam, Dipkamal Bhusal, Niyati Shah, and Nidhi Rastogi. Ctibench: A benchmark for evaluating LLMs in cyber threat intelligence. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

[7] Eitan Altman. *Constrained Markov Decision Processes*. Chapman and Hall/CRC, 1999.

[8] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. In *Proceedings of the 15th Annual Conference on Computational Learning Theory (COLT)*, pages 332–347, 2002.

[9] Andrew G. Barto, Richard S. Sutton, and Charles W. Anderson. Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-13(5):834–846, 1983.

[10] Marc G. Bellemare, Will Dabney, and Rémi Munos. A distributional perspective on reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2017.

[11] Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.

[12] Steven J. Bradtke and Andrew G. Barto. Linear least-squares algorithms for temporal difference learning. *Machine Learning*, 22(1–3):33–57, 1996.

[13] Ionuţ Cardei. Instruction fine tuning dataset design. Technical report, Athena Labs, Athena Security Group, 2025. Internal design document describing the Athena CTI Neo4j graph schema and template-based IFT dataset generation.

[14] Dongkyu Derek Cho, Huan Song, Arijit Ghosh Chowdhury, Haotian An, Yawei Wang, Rohit Thekkanal, Negin Sokhandan, Sharlina Keshava, and Hannah Marlowe. Breaking the safety-capability tradeoff: Reinforcement learning with verifiable rewards maintains safety guardrails in LLMs. *arXiv preprint arXiv:2511.21050*, 2025.

[15] Paul Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *arXiv preprint arXiv:1706.03741*, 2017. NeurIPS 2017.

[16] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Eric Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. Scaling instruction-finetuned language models. *arXiv preprint arXiv:2210.11416*, 2022.

[17] DeepSeek-AI. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[18] DeepSeek-AI, Daya Guo, Dejian Yang, et al. Deepseek-r1: Incentivizing reasoning capability in LLMs via reinforcement learning. *Nature*, 645:633–638, 2025. Also available as arXiv:2501.12948.

[19] Marc Peter Deisenroth and Carl Edward Rasmussen. PILCO: A model-based and data-efficient approach to policy search. In *International Conference on Machine Learning (ICML)*, 2011.

[20] Damien Ernst, Pierre Geurts, and Louis Wehenkel. Tree-based batch mode reinforcement learning. *Journal of Machine Learning Research*, 6:503–556, 2005.

[21] Clemens Fieblinger, A Min Tjoa, et al. Actionable cyber threat intelligence using knowledge graphs and large language models. *arXiv preprint*, 2024.

[22] Scott Fujimoto, David Meger, and Doina Precup. Off-policy deep reinforcement learning without exploration. In *International Conference on Machine Learning (ICML)*, 2019.

[23] Javier García and Fernando Fernández. A comprehensive survey on safe reinforcement learning. *Journal of Machine Learning Research*, 16:1437–1480, 2015.

[24] John C. Gittins. Bandit processes and dynamic allocation indices. *Journal of the Royal Statistical Society: Series B*, 41(2):148–177, 1979.

[25] Daya Guo, Dejian Yang, Haowei Zhang, et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs through reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[26] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning (ICML)*, 2018.

[27] Danijar Hafner, Timothy Lillicrap, Mohammad Norouzi, and Jimmy Ba. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations (ICLR)*, 2020.

[28] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence (AAAI)*, 2018.

[29] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. *arXiv preprint arXiv:2103.03874*, 2021.

[30] Matteo Hessel, Joseph Modayil, Hado van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Azar, and David Silver. Rainbow: Combining improvements in deep reinforcement learning. In *Proceedings of the 32nd AAAI Conference on Artificial Intelligence (AAAI)*, 2018.

[31] Ronald A. Howard. *Dynamic Programming and Markov Processes*. The MIT Press, 1960.

[32] H. Hu, P. Zhang, et al. Seceval: A comprehensive benchmark for evaluating cybersecurity knowledge in foundation models. *arXiv preprint*, 2024.

[33] Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine. When to trust your model: Model-based policy optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[34] Hangyuan Ji, Jian Yang, Linzheng Chai, Chaoren Wei, Liqun Yang, Yunlong Duan, Yunli Wang, Tianzhen Sun, Hongcheng Guo, Tongliang Li, et al. Sevenllm: Benchmarking, eliciting, and enhancing abilities of large language models in cyber threat intelligence. *arXiv preprint arXiv:2405.03446*, 2024.

[35] Yuxin Jiang, Yufei Wang, Qiyuan Zhang, Xingshan Zeng, Liangyou Li, Jierun Chen, Chaofan Tao, Haoli Bai, and Lifeng Shang. From verifiable dot to reward chain: Harnessing verifiable reference-based rewards for reinforcement learning of open-ended generation. *arXiv preprint arXiv:2601.18533*, 2026.

[36] Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, et al. Kimi k1.5: Scaling reinforcement learning with LLMs. *arXiv preprint arXiv:2501.12599*, 2025.

[37] Vijay R. Konda and John N. Tsitsiklis. Actor-critic algorithms. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2000.

[38] Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative Q-learning for offline reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[39] Michail G. Lagoudakis and Ronald Parr. Least-squares policy iteration. *Journal of Machine Learning Research*, 4:1107–1149, 2003.

[40] Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, Yuling Gu, Saumya Malik, Victoria Graf, Jena D. Hwang, Jiangjiang Yang, Ronan Le Bras, Oyvind Tafjord, Chris Wilhelm, Luca Soldaini, Noah A. Smith, Yizhong Wang, Pradeep Dasigi, and Hannaneh Hajishirzi. Tulu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.

[41] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C. H. Hoi. CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. *arXiv preprint arXiv:2207.01780*, 2022. NeurIPS 2022.

[42] Matan Levi, Yair Allouche, Daniel Ohayon, and Anton Puzanov. Cyberpal.ai: Empowering LLMs with expert-driven cybersecurity instructions. *arXiv preprint arXiv:2408.09304*, 2024.

[43] Matan Levi, Daniel Ohayon, Ariel Blobstein, Ravid Sagi, Ian Molloy, and Yair Allouche. Toward cybersecurity-expert small language models. *arXiv preprint arXiv:2510.14113*, 2025.

[44] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.

[45] Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015. ICLR 2016.

[46] Zefang Liu, Jialei Shi, and John F. Buford. Cyberbench: A multi-task benchmark for evaluating large language models in cybersecurity. *arXiv preprint arXiv:2407.08665*, 2024.

[47] R. McMillan. Defining threat intelligence. Technical report, Gartner, 2013.

[48] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2016.

[49] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[50] Youssef Mroueh. Reinforcement learning with verifiable rewards: GRPO's effective loss, dynamics, and success amplification. *arXiv preprint arXiv:2503.06639*, 2025.

[51] Andrew Y. Ng and Stuart J. Russell. Algorithms for inverse reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2000.

[52] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155*, 2022.

[53] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155*, 2022.

[54] Romain Paulus, Caiming Xiong, and Richard Socher. A deep reinforced model for abstractive summarization. *arXiv preprint arXiv:1705.04304*, 2017. Often cited in the context of RL training for sequence generation; appears in ICLR-era literature.

[55] Romain Paulus, Caiming Xiong, and Richard Socher. A deep reinforced model for abstractive summarization. *arXiv preprint arXiv:1705.04304*, 2017. ICLR 2018.

[56] Marc'Aurelio Ranzato, Sumit Chopra, Michael Auli, and Wojciech Zaremba. Sequence level training with recurrent neural networks. *arXiv preprint arXiv:1511.06732*, 2015. ICLR 2016.

[57] Herbert Robbins. Some aspects of the sequential design of experiments. *Bulletin of the American Mathematical Society*, 58(5):527–535, 1952.

[58] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *International Conference on Learning Representations (ICLR)*, 2016.

[59] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, and David Silver. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020.

[60] John Schulman, Sergey Levine, Philipp Moritz, Michael I. Jordan, and Pieter Abbeel. Trust region policy optimization. In *International Conference on Machine Learning (ICML)*, 2015.

[61] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[62] Amulya Setlur et al. Rewarding progress: Scaling automated process verifiers for LLM reasoning. *arXiv preprint arXiv:2410.08146*, 2024.

[63] Rulin Shao, Shuyue Stella Li, Rui Xin, Scott Geng, Yiping Wang, Sewoong Oh, Simon Shaolei Du, Nathan Lambert, Sewon Min, Ranjay Krishna, Yulia Tsvetkov, Hannaneh Hajishirzi, Pang Wei Koh, and Luke Zettlemoyer. Spurious rewards: Rethinking training signals in RLVR. *arXiv preprint arXiv:2506.10947*, 2025.

[64] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[65] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.

[66] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of go without human knowledge. *Nature*, 550(7676):354–359, 2017.

[67] B. F. Skinner. *The Behavior of Organisms: An Experimental Analysis*. Appleton-Century, 1938.

[68] Mingyang Song, Mao Zheng, Zheng Li, Wenjie Yang, Xuan Luo, Yue Pan, and Feng Zhang. Fastcurl: Curriculum reinforcement learning with stage-wise context scaling for efficient training R1-like reasoning models. *arXiv preprint arXiv:2503.17287*, 2025.

[69] Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel M. Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul Christiano. Learning to summarize from human feedback. *arXiv preprint arXiv:2009.01325*, 2020. NeurIPS 2020.

[70] Blake E. Strom, Andy Applebaum, Doug P. Miller, Kathryn C. Nickels, Adam G. Pennington, and Cody B. Thomas. MITRE ATT&CK: Design and philosophy. Technical Report MP180360, The MITRE Corporation, 2018.

[71] Yi Su, Dian Yu, Linfeng Song, Juntao Li, Haitao Mi, Zhaopeng Tu, Min Zhang, and Dong Yu. Crossing the reward bridge: Expanding RL with verifiable rewards across diverse domains. *arXiv preprint arXiv:2503.23829*, 2025.

[72] Yi Su, Dian Yu, Linfeng Song, Juntao Li, Haitao Mi, Zhaopeng Tu, Min Zhang, and Dong Yu. Expanding RL with verifiable rewards across diverse domains. *arXiv preprint arXiv:2503.23829*, 2025.

[73] Richard S. Sutton. Learning to predict by the methods of temporal differences. *Machine Learning*, 3(1):9–44, 1988.

[74] Richard S. Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. In *Proceedings of the 7th International Conference on Machine Learning (ICML)*, pages 216–224, 1990.

[75] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.

[76] Richard S. Sutton, David A. McAllester, Satinder P. Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2000.

[77] William R. Thompson. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, 25(3/4):285–294, 1933.

[78] Edward L. Thorndike. Animal intelligence: An experimental study of the associative processes in animals. *Psychological Review: Monograph Supplements*, 2(4), 1898.

[79] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the 30th AAAI Conference on Artificial Intelligence (AAAI)*, 2016.

[80] Longwen Wang, Xuan'er Wu, Xiaohui Hu, Yirui Liu, Yuankai Fan, Kaidong Yu, Qizhen Weng, Wei Xi, and Xuelong Li. VeRPO: Verifiable dense reward policy optimization for code generation. *arXiv preprint arXiv:2601.03525*, 2026.

[81] Christopher J. C. H. Watkins. *Learning from Delayed Rewards*. PhD thesis, King's College, University of Cambridge, 1989.

[82] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992.

[83] Chenxing Wei, Jiarui Yu, Ying Tiffany He, Hande Dong, Yao Shu, and Fei Yu. ReDit: Reward dithering for improved LLM policy optimization. *arXiv preprint arXiv:2506.18631*, 2025.

[84] Jason Wei, Maarten Bosma, Vincent Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V. Le. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations (ICLR)*, 2022.

[85] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*, 2022.

[86] Xumeng Wen, Zihan Liu, Shun Zheng, et al. Reinforcement learning with verifiable rewards implicitly incentivizes correct reasoning in base LLMs. *arXiv preprint arXiv:2506.14245*, 2025.

[87] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256, 1992.

[88] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256, 1992.

[89] X. Wu, L. Wang, et al. Open-cykg: Constructing a cyber threat intelligence knowledge graph from open sources. *arXiv preprint*, 2022.

[90] Lecheng Yan, Ruizhe Li, Guanhua Chen, Qing Li, Jiahui Geng, Wenxi Li, Vincent Wang, and Chris Lee. Spurious rewards paradox: Mechanistically understanding how RLVR activates memorization shortcuts in LLMs. *arXiv preprint arXiv:2601.11061*, 2026.

[91] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, et al. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

[92] Y. Yu, J. Kang, et al. Ctikg: A large-scale, high-quality cyber threat intelligence knowledge graph. *arXiv preprint*, 2024.

[93] Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? *arXiv preprint arXiv:2504.13837*, 2025.

[94] Sheng Zhang, Qianchu Liu, Guanghui Qin, Tristan Naumann, and Hoifung Poon. Med-RLVR: Emerging medical reasoning from a 3B base model via reinforcement learning. *arXiv preprint arXiv:2502.19655*, 2025.

[95] Y. Zhang, H. Chen, et al. Cyberkg: A cyber threat intelligence knowledge graph for threat analysis. *arXiv preprint*, 2023.

[96] Z. Zhao, H. Li, et al. K-ctiaa: Knowledge graph enhanced cyber threat intelligence action analysis. *arXiv preprint*, 2024.

[97] Brian D. Ziebart, Andrew Maas, J. Andrew Bagnell, and Anind K. Dey. Maximum entropy inverse reinforcement learning. In *Proceedings of the 23rd AAAI Conference on Artificial Intelligence (AAAI)*, 2008.

[98] Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B. Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*, 2019.